R65 Help Booklet

Contents

animate

```
ANIMATE

func expaint:boolean;
  Handles toggle graphics.
  Provides code to paint and apply motion.
  Returns true if animation loop should be
  Stops if motion detects stop
  (e.g. expaint:=false)

func exkey(ch:char):boolean;
  Provides code to handle keys pressed.
  Returns true if animation loop should be
  stopped because of key pressed.
  (e.g. exkey:=ch=ord(0);)

Usage:
{I IANIMATE:P}

animate(autorepeat)
  Run the animation loop.
  Set autorepeat to true if cursor key repetion
  should start immediately, not after short
  delay.
```

arglib

```
LIBRARY ARGLIB;

MEMORY
    NUMARG   = $005f: integer&;
    ARGLIST  = $0060: array[10] of integer;
    ARGLISTS = $0060: array[63] of char&;
    ARGTYPE  = $00a0: array[31] of char&;
    FILFLG   = $00da: integer&;
    FILDRV   = $00dc: integer&;
    FILCYC   = $0311: integer&;
    FILCY1   = $0330: integer&;
    FILNAM   = 0301: array[15] of char&;
    FILNM1   = $0320: array[15] of char&;
    FILSTP   = $0312: char&;

VARIABLES
      _carg: integer;

ROUTINES
proc _argerror(e: integer);
proc _agetval(var value:integer;
        var default:boolean);
proc _agetstring(var string: array[15] of char;
        var default: boolean;
        var cyclus, drive: integer);
```

compile

```
COMPILE name[.cy[,drv]] [/option]
  name:    file name of program to compile
  option:  p: no hard copy print
           l: put line info in object code
           r: range checking for index
           n: no loader file
           f: force compile
  Default drive is 1.
  The /f option forces a compilation even if there is
  a object file with equal or higher version on
  disk 0 or 1.

Compiler directives:
  {$I filename}   include file
  {$R-) {$R+}     range checking, default false
  {$U-} {$U+}     unique identifiers in scope
```

dir

```
DIR [drive] [name] [/s]

  where  /s        Sort directory
         drive     Drive number, default 1
         name      name of a disk

  Examples  DIR               DIR /S
            DIR 0             DIR 0 /S
            DIR PSOURCE       DIR PSOURCE /S
```

edit

```
EDIT name[.cy[,drv]]
  name:   name of Pascal source file
          can include a subtype :X

  default drive is 1
  default for subtype is :P

  Edit a sequential file with the external editor.
```

errors

```
System Error Codes
==================
01      Read/write error
02      Checksum error
03      Escape exit during read/write
04      Record number error
05      File type error
06      File not found
07      Disk not ready
08      Directory full, file not stored
09      Illegal IRQ
10      Expression missing
11      Memory cell cannot be changed
12      Break table full, not inserted
13      Illegal memory cell for break
14      Double breakpoint setting
15      End of line expected
16      Syntax wrong in register name or =
17      Breakpoint not found in table
18      Syntax wrong in store
19      File subtype wrong or missing
20      Wrong file type, not runnable
21      Unknown monitor command
22      Illegal opcode for STEP/TRACE
23      Too many open files, not opened
24      Direction error in sequential read/write
25      Wrong file number, file not open
26      Disk full, file not stored
27      Random access index out of range
28      Illegal drive for random access
29      File not open for random access write

EXDOS Error Codes
=================
61      Wildcard not allowed
62      Only for disk, not tape
63      Illegal copy
64      File too large
65      Write error
66      Import error
67      Unknown emulator command
68      Unable to run external editor
69      File not changed or no new file
```

```
Pascal Runtime Error Codes
==========================
81      Division by zero
82      Stack overflow
83      Index out of bounds
84      Program not found or not a Pascal program
85      Illegal P-code
86      Escape during execution
87      No loader file made by compiler
88      Heap overflow
89      Pointer not allocated (NIL)
90      Writing to constant string not allowed
91      String too long (more than 63 characters)
92      Only the last allocated string can be released

Pascal Argument Error Codes
===========================
101     Argument is not string or default
102     Argument is not number or default
103     Argument is not starting with /
104     Unknown argument
105     Drive is not 0 or 1
106     Argument syntax error
107     Too many arguments
```

export

```
EXPORT name[.cy[,drv]]
  name:   name of Pascal source file
          can include a subtype :X

  default drive is 1
  default for subtype is :P

  Export a sequential file.
```

filelib

```
LIBRARY FILELIB;

MEMORY
    FILFLG = $00da: integer&;
    FILDRV = $00dc: integer&;
    FILTYP = $0300:char&;
    FILNAM = $0301: array[15] of char&;
    FILCYC = $0311: integer&;
    FILSTP = $0312: char&;
    FILNM1 = $0320: array[15] of char&;
    FILCY1 = $0330: integer&;

ROUTINES
func _uppercase(ch1: char): char;
proc _asetfile(name: array[15] of char;
      cyclus,device: integer; subtype: char);
func _freedrv(drive:integer; printit:boolean);
```

graph

```
GRAPH filename
  display the graph of function values
  stored in table on internal display

TGRAPH filename
  display the graph of function values
  stored in table on Tektronix 4010

MAKETABLE filename
  make a table for graph/tgraph
```

graphics

```
LOW RESOLUTION GRAPHICS ON THE R65 COMPUTER

Graphics can be displayed in split mode in a
graphic overlay, or in fullscreen mode.

To toggle between full screen and split mode
use the key "ctrl-l".

To clear the graphics canvas and release the
graphics memory use the command "GREND".
```

help

```
HELP topic

  where topic is a command name or another
  file with extention :H on drive 1 or 0.
  Lists available topics if topic not found.
```

import

```
IMPORT name[.cy[,drv]]
  name:   name of Pascal source file
          can include a subtype :X

  default drive is 1
  default for subtype is :P

  Import a sequential file
```

iotest

```
PROGRAM IOTEST
USES SYSLIB, STRLIB
VAR S1: CPNT
    FOUT: FILE
BEGIN
  WRITELN ('IOTEST')
  STRFIO('IOTEST:B',1)
  OPENW(FOUT)
  WRITELN(@FOUT, 'IOTEST')
  CLOSE(FOUT)
END.
```

keys

```
IMPORTANT KEYBOARD KEYS ON THE R65 SYSTEM

CTRL-l          split/full screen graphics
CTRL-r          print all on
CTRL-t          print all off

CTRL-<return>   clear to end of screen
CTRL-<right>    insert char
CTRL-<left>     delete char
CTRL-<down>     scroll down
CTRL-<up>       scroll up
SHIFT-CTRL      is required on Mac/UTM/Debian
```

ladders

```
LADDERS [/D]
game with ladders

    /D endless demo mode
```

lastvers

```
LASTVERS name[.cy[,drv]] [/option]
  name:   name of Pascal source file
        option: /f: force compilation
  default drive is 1

  This is a helper program for COMPILE. It rejects
  the COMPILE command if an object file (:R) exists
  on one of the two drives with a equal or higher
  version than the source. In that case compilation
  is aborted unless it is forced with the /f option.
```

ledlib

```
LIBRARY LEDLIB;

MEMORY
      LEDREG=$1432: array[7] of char&;

ROUTINES
proc _ledstring(s:cpnt);
proc _ledstop;
proc _ledhex(d,p,digits: integer);
proc _ledbyte(d: integer);
```

mathlib

```
LIBRARY MATHLIB;

CONSTANTS
      PI = 3.14159;
      E  = 2.71828;

ROUTINES
func fabs(x:real):real;
func sqrt(n:real):real;
func cos(x:real):real;
func sin(x:real):real;
func tan(x:real):real;
proc writeflo(f:file;r:real);
proc writef0(f:file;d:integer;
    r:real;fl:integer;centered:boolean);
proc writefix(f:file;d:integer;r:real);
func readflo(f:file):real;
func ln(r:real):real;
func exp(x:real):real;
func log(x:real):real;
```

nroff

```
NROFF(1)


NAME
====
NROFF - text formatter for R65

SYNOPSIS
========
NROFF filnam[:sub[.drv]] [linewidth]
Default line width is 40.

DESCRIPTION
===========
NROFF formats plain text files using simple dot
commands. Lines starting with a dot are
formatting requests.All other lines are normal
text and are formatted using fill mode.

TEXT MODES
==========
Fill modeWords are wrapped to fit the current
line width.
No fill modeText is printed exactly as entered.
.fi     Enable fill mode.
.nf     Enable no fill mode.

HEADINGS
========
.TH     Manual header line.
.SH     Section heading. Uses underline or
inverse video depending on width.
.SS     Subsection heading.

PARAGRAPHS
==========
.PP     Start new paragraph.
.br     Line break.
.sp [n] Insert blank lines.

INDENTATION
===========
```

```
.RS     Increase indentation.
.RE     Restore indentation.
.IP labelIndented paragraph. Label followed by
text with hanging indent.

TABS
====
.ta n1 n2 ...Set tab stops.
Tabs are expanded during output.
If no tabs are defined, default spacing is 8
cols.

TEXT STYLE
==========
.B text Bold line. On 40 columns inverse video
is used.

INPUT RULES
===========
Maximum input line length depends on buffer
size.
Control characters except TAB are ignored.

OUTPUT WIDTH
============
Line width may be given as optional parameter.
Example NROFF MANUAL:B,1 80
Widths above 40 use printer style headings.

TABLES
======
Tables are created using no fill mode and tabs.
Example:
COL1    COL2    COL3
====    ====    ====
AA      BBB     CCCC

LIMITATIONS
===========
No macros or registers.
No escape sequences.
Formatting is single pass.

AUTHOR
======
```

Written for the R65 system.
Implementation by R. Richarz and ChatGPT.

END
===
End of NROFF manual.

paint

```
PAINT filename[.cyclus][,drive]

  Painting on the R65 low resolution graphics
  canvas

  arrows    move cursor
  C         clear canvas
  M         switch color
  p         paint a dot at cursor position
  L         drawing line mode
  R         drawing rectange mode
  S         drawing character mode
  return    draw object and exit mode
  esc       exit mode without drawing
  W         write canvas to disk
  Q         write canvas to disk, quit
  K         kill program without writing
```

pcodes

```
PCODES filname[disk,from,to]

  Display P-codes of a Pascal Program.
  Pascal source code and object code must be
  on the same drive.

  If the program has been compiled with the /L
  option (line numbers), pcode displays the
  source line together with the pcodes for this
  line, and pcodes of a specified range of
  lines (from/to) can be displayed. Use SETB
  and RESETB to set breakpoints.
```

pedit

```
PEDIT filename

 ESC              Enter command
 t          Go to top of file
 b          Go to bottom of file
 l n        Go to line n
 f          Find
 a          Find again
 z          Clear marks
 c n        Mark lines of copy
 p          Paste copied lines
 m          Move copied lines
 d n        Delete n lines
 w          Write file
 q          Write file and quit
 k          Kill (quit without writing)
 ?,h        Help
```

plotlib

```
LIBRARY PLOTLIB;

CONSTANTS
      XSIZE=223;
      YSIZE=117;
      XWORDS=28;
      WHITE=0;
      INVERSE=1;
      BLACK=2;
      PLOTDEV=@128;

MEMORY
      KEYPRESSED=$1785: char&;

VARIABLES
      _xcursor, _ycursor: integer;

ROUTINES
proc _delay10msec(time:integer);
func _syncscreen;
proc _grinit;
proc _grend;
proc _cleargr;
proc _fullview;
proc _splitview;
proc _plot(x,y,c:integer);
proc _move(x,y:integer);
proc _draw(x,y,c:integer);
proc _plotmap(x,y,m:integer);
proc _waitforKEY;
```

R65

```
R65 QUICK REFERENCE
===================

MONITOR COMMANDS
================
DIR     Directory listing.
COPY    Copy file between drives.
DELETE  Delete a file.
EDIT    Edit text file.
LOAD    Load file to memory.
STORE   Store memory sektion to disk.
GO      Execute machine code in memory.
DIS     Disassemble memory.
REG     Show CPU registers.
DATE    Show or set date.
FLOPPY  Select disk image.
IMPORT  Import host file.
EXPORT  Export file to host.
RUN     Run a maschine code program.

PASCAL
======
COMPILE Compile Pascal program.
EDIT    Edit a text file.
name    Execute Pascal program name.
LIB     Load Pascal library.

NROFF
=====
NROFF   Format text document.
.TH     Manual header.
.SH     Section heading.
.SS     Subsection heading.
.PP     New paragraph.
.IP     Indented paragraph.
.nf / .fiNo fill / fill mode.
.ta     Set tab stops.

SCREEN CONTROL
==============
CT-E    Clear line.
CT-RTN  Clear to end of display.
```

```
CT-RGT  Insert character.
CT-LFT  Delete character.
CTRL-A  Cursor home.
CT-UP   Roll up.
CT-DN   Roll down.
```

ralib

```
LIBRARY RALIB;

CONSTANTS
      FREAD=$00;
      FWRITE=$20;
      FNEW=$30;

ROUTINES
func _uppercase(ch:char):char;
func _attach(fname:array[15] of char;
  cyclus,drive,operation,size,start:integer;
  subtype:char):file;
func _getsize:integer;
proc _getword(device:file; address:integer;
  var word:integer);
proc _putword(device:file; address:integer;
  word:integer);
proc _getreal(device:file; address:integer;
  var rvalue:array[1] of %integer);
proc _putreal(device:file; address:integer;
  rvalue:array[1] of %integer);
```

striolib

```
LIBRARY STRIOLIB;

CONSTANTS
    NAMESIZE = 15;

MEMORY
    filerr  =$00db: integer&;

VARIABLES
      scarg: integer;

ROUTINES
proc _argerror(e: integer);
proc _checkfilerr;
proc _sgetval(var value:integer;
                     var default:boolean);
proc _sgetstring(string: cpnt; var default: boolean;
                 var cyclus, drive: integer);
proc _ssetsubtype(sname: cpnt; subtype:char;
                  force: boolean);
proc _strfio(name:cpnt; drive: integer);
proc _getdiskname(s: cpnt; drive: integer);
proc _change_disk(s: cpnt; drv: integer);
proc srunprog(name: cpnt; cyc: integer; drv: integer);
```

strlib

```
LIBRARY STRLIB;

CONSTANTS
      STRSIZE=64;
      ENDMARK=chr(0);

ROUTINES
proc _runerr(e:integer);
func _new: cpnt;
proc _release(s: cpnt);
func _strlen(strin:cpnt):integer;
proc _strcpy(strin, strout:cpnt);
proc _stradd(strin,strinout:cpnt);
func _strcmp(s1,s2:cpnt):integer;
func _strpos(ch:char; s1:cpnt;
          start:integer): integer;
func _strread(f:file; s:cpnt;
                 var ateof0:boolean):integer;
proc _hexstr(d:integer; s:cpnt);
```

syslib

```
LIBRARY SYSLIB;

CONSTANTS
  TAB8     = chr(9);
  HOM      = chr(1);
  CSC      = chr($11);
  LF       = chr($a);
  FF       = chr($c);
  CR       = chr($d);
  EOF      = chr($7f);
  NORVID   = chr($0b);
  INVVID   = chr($0e);
  PRTON    = chr($12);
  PRTOFF   = chr($14);
  MMAXSEQ  = 8;
  TOPMEM   = $c780;
  MAXINT   = $7fff;
  INPUT    = @0;
  OUTPUT   = @0;
  KEY      = @1;
  PRINTER  = @1;

MEMORY
  RUNERR   = $000c: integer&;
  ENDSTK   = $000e: integer;
  BUFFPN   = $0015: integer&;
  IOCHECK  = $0023: boolean&;
  CURPOS   = $00ee: integer&;

VARIABLES
  _day:    integer;
  _month:  integer;
  _year:   integer;

ROUTINES
proc _setemucom(i:integer);
func _getbcd(address: integer): integer;
proc _getdate;
proc _prtdate(device: file);
func _abs(x: integer): integer;
func _mod(x,n: integer): integer;
proc _tab(pos: integer);
proc _abort;
```

```
proc _prtext8(device: file;
            text: array[7] of char);
proc _prtext16(device: file;
            text: array[15] of char);
func _random: integer;
```

table

```
graph
  display the graph of function values
  stored in table
table
  make a table for display with graph
```

teklib

```
LIBRARY TEKLIB;

CONSTANTS
      MAXX = 1023;
      MAXY = 780;
      MAXCOLUMNS = 74;
      MAXLINES   = 35;
      SOLID      = 1;
      DOTTED     = 2;
      DOTDASH    = 3;
      SHORTDASH  = 4;
      LONGDASH   = 5;
      PLOTTER    = @1;

VARIABLES
      _xs,_ys: integer;

ROUTINES
proc _clearscreen;
proc _starttek;
proc _endtek;
proc _startdraw(x1,y1:integer);
proc _draw(x2,y2:integer);
proc _enddraw;
proc _drawvector(x1,y1,x2,y2:integer);
proc drawrectange(x1,y1,x2,y2:integer);
proc _moveto(x1,y1: integer);
proc _delay10msec(time:integer);
proc _setchsize(size:integer);
proc _setlinemode(type:integer);
```

timelib

```
LIBRARY TIMELIB;

VARIABLES
      tenmillis,seconds,minutes,hours,
    difftenmillis: integer;

ROUTINES
proc gettime;
proc prttime(device: file);
func timediff: real;
```

view

```
VIEW filename

 ESC, q         Quit
 CUP, CDOWN     Scroll up/down
 CLEFT, CRIGHT  Scroll 12 lines up/down
 /, f           find string
 n              next hit
 ?, h           help
```

wildlib

```
LIBRARY WILDLIB;

CONSTANTS
      NAMESIZE   = 15;
      NUMENTRIES = 255;

MEMORY
      FILNAM=$0301: array[NAMESIZE] of char&;

ROUTINES
proc _test(s1:array[NAMESIZE] of char;
      var found:boolean);
proc _findentry(var nm:array[NAMESIZE] of char;
       drv:integer;var ent: integer;
       var fnd,lst:boolean);
proc _sfindentry(name: cpnt; drv:integer;
       var ent: integer;
       var fnd,lst: boolean);
proc _writename(nm1:array[15] of char);
```